DFA Minimizer

Doodlebob

The following is an auto-explainer for DFA Minimization as represented by a series of figures depicting the evolving partitions of the DFAâ€™s state set.

A Deterministic Finite Automaton (DFA) is denoted by the quintuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

and the minimization process works by iteratively refining partitions of the state set until all equivalent states are grouped together. In the minimization process:

- The initial partition divides Q into two blocks: the final states F and the non-final states $Q \setminus F$.
- At each subsequent step, the partition is refined by examining the transitions for every input symbol and grouping together states that behave equivalently.
- This iterative refinement continues until no further splitting is possible, thus yielding a minimal DFA.

1 DFA Definition and Minimization Overview

A Deterministic Finite Automaton is defined as a 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where:

- Q is a finite set of states,
- Σ is a finite input alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the start state, and
- $F \subseteq Q$ is the set of final (accepting) states.

The minimization of a DFA entails computing an equivalent DFA with the fewest possible states. The process begins by partitioning the state set into final and non-final states and then repeatedly refining these partitions by identifying states that are indistinguishable with respect to the input symbols. The resulting minimized DFA retains the same language as the original.

2 DFA Specification

We will use the DFA shown below to illustrate the minimization process. As we work through the algorithm to determine which pairs of states are indistinguishable from each other, we will fill in the Indis Table. Note that the table only includes the cells below the diagonal, because we only need one cell comparing p and q (the order is irrelevant) and there is no need to compare a state with itself.

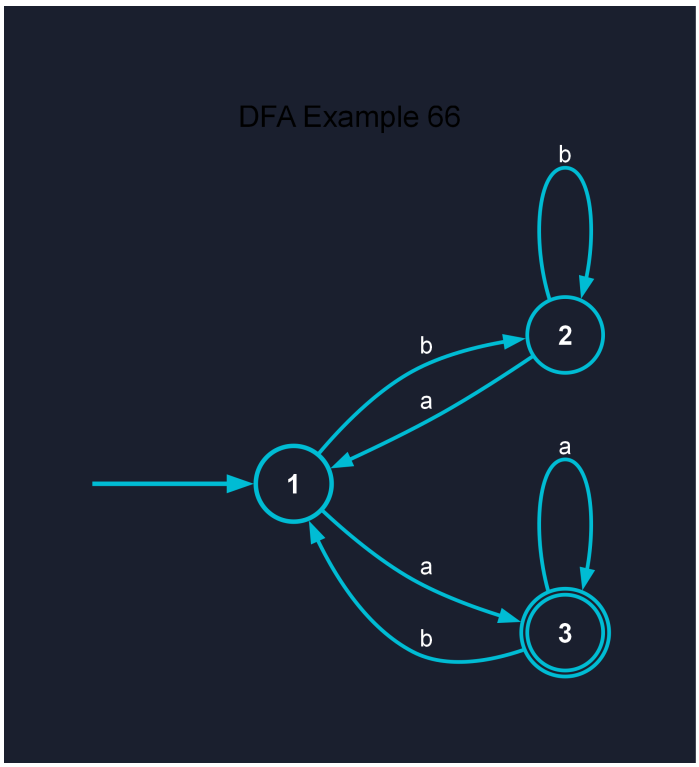

Figure 1: The original DFA to be minimized

Transition Function

State	Input Symbols	
	a	**b**
1	3	2
2	1	2
3	3	1

The set of accepting states is $F = \{3\}$.

Understanding the Indis Table:

- **Empty cells** correspond to a `True` value, meaning we have not yet learned that the pair of states are distinguishable. These pairs are *potentially indistinguishable*.
- **Cells marked with "X"** correspond to a `False` value, indicating we have determined that the pair of states *are* distinguishable. There exists at least one input string that leads to different outcomes (one accepts, one rejects) from these two states.
- The table only shows cells below the diagonal because:
  - $\text{Indis}(p, q) = \text{Indis}(q, p)$ (order doesn't matter)
  - $\text{Indis}(p, p)$ is always `True` (a state can't be distinguished from itself)

3 Indistinguishability Algorithm

Initialization

The algorithm implements the base cases in the definition of distinguishable states by setting the $\text{Indis}(p, q)$ value to `False` for those pairs of states that are an accepting state paired with a non-accepting state.

Why? If one state is accepting and the other is non-accepting, they are immediately distinguishable: the empty string ε is accepted from one state but rejected from the other. This is the simplest way two

states can differ. Since states 3 are accepting and all other states are non-accepting, we fill in the rows

Pair	**State** p	**State** q	**Indis**(p, q) **Value**
$(1, 2)$	non-accepting	non-accepting	`True` (leave empty)
$(1, 3)$	non-accepting	accepting	`False` (mark with "X")
$(2, 3)$	non-accepting	accepting	`False` (mark with "X")

Table 1: Initialization: Setting Indis(p, q) for all 3 state pairs

and columns for 3.

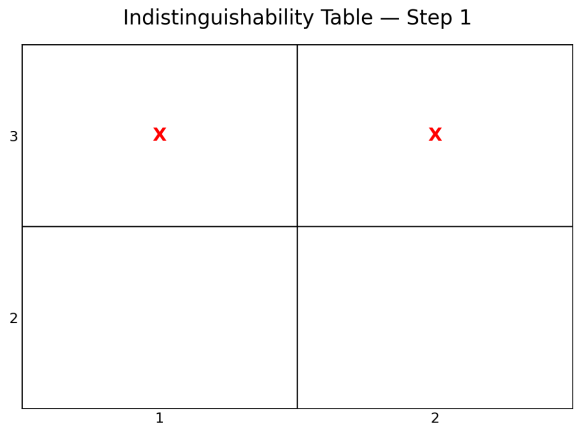

Figure 2: Indis Table afterinitialization.

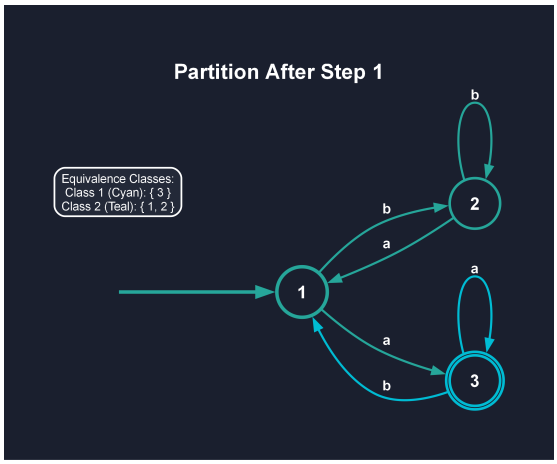

Figure 3: Current equivalence partition afterinitialization. States with the same color are potentially indistinguishable.

Current equivalence classes:

$$\{\{3\}, \{2, 1\}\}$$

Iteration 1

We now scan through all empty cells (i.e., cells that still have `True` values). For this iteration, that means we have 1 cells to process.

Strategy: For each unmarked pair (p, q), we test all input symbols. If we find *any* input a where $\delta(p, a)$ and $\delta(q, a)$ lead to states that are already marked as distinguishable, then (p, q) must also be distinguishable.

$(1, 2)$ We now consider both possible inputs to see if these two states are distinguishable given what we know so far.

- **Input a:** Here we see that $\delta(1, a)$ leads to state 3 and $\delta(2, a)$ leads to state 1. Since Indis$(1, 3)$ is `False`, we change Indis$(1, 2)$ to `False`.

  $\rightarrow$ We set Indis$(1, 2)$ = False

  Explanation: If states 1 and 3 are distinguishable (we already marked them), and we can reach them from states 1 and 2 using input a, then states 1 and 2 must also be distinguishable. Any string that distinguishes 1 from 3 can be prepended with a to distinguish 1 from 2.

`plots/pair_steps/iter1_paira_input1.png`

Figure 4: Visual analysis for pair $(1, 2)$: On input a, transitions lead to the already-marked pair $(1, 3)$, forcing Indis$(1, 2)$ to become `False`.

This iteration through all empty cells caused **1** cell value(s) to change from `True` to `False`. Thus, at the end of this iteration, the table looks like this, with updated values shown in red "X" marks:

Progress check: Since we made changes, we must continue to the next iteration. The newly marked pairs might help us identify additional distinguishable pairs in the next round.

Current equivalence classes:

$$\{\{1\}, \{2\}, \{3\}\}$$

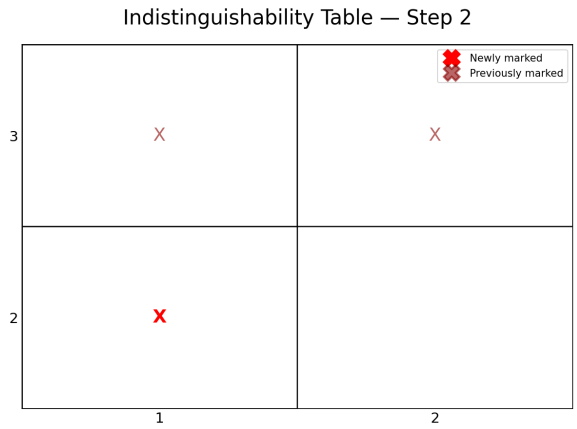

Figure 5: Indis Table afteriteration 1. Bright red "X": newly marked; dark red "X": previously marked.

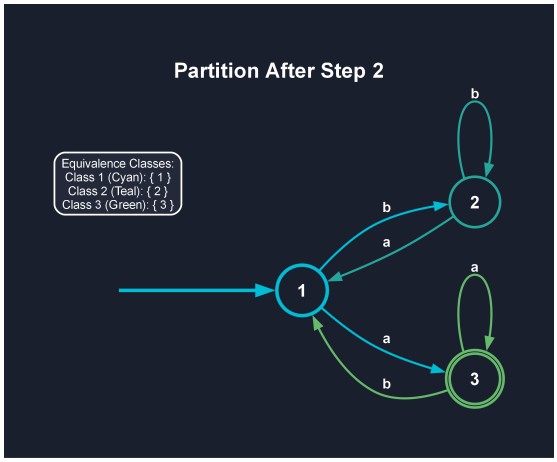

Figure 6: Current equivalence partition afteriteration 1. States with the same color are potentially indistinguishable.

Iteration 2

We

Strategy: For each unmarked pair (p, q), we test all input symbols. If we find *any* input a where $\delta(p, a)$ and $\delta(q, a)$ lead to states that are already marked as distinguishable, then (p, q) must also be distinguishable. This pass through the 0 remaining cells did not cause a change. We now break out of our loop and we are finished finding all indistinguishable pairs of states.

Why stop? If a full iteration produces no new markings, it means we've found all distinguishable pairs. The remaining empty cells represent states that truly cannot be distinguished by any input string — these are the indistinguishable pairs we will merge.

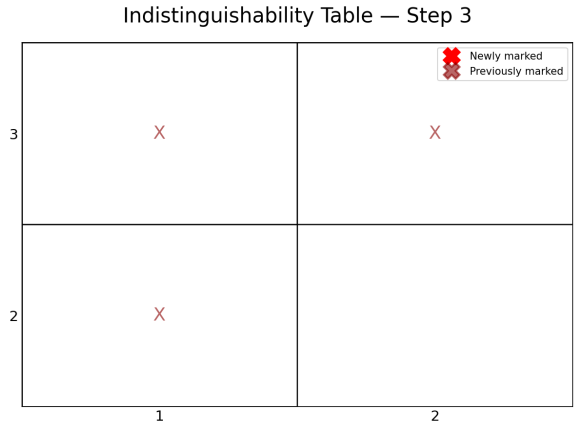

Figure 7: Indis Table afteriteration 2. Bright red "X": newly marked; dark red "X": previously marked.

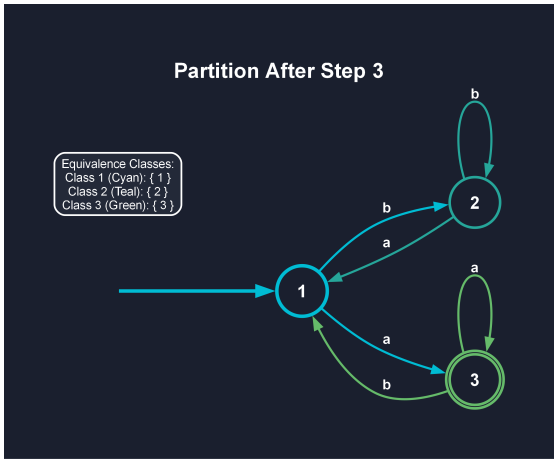

Figure 8: Current equivalence partition afteriteration 2. States with the same color are potentially indistinguishable.

Current equivalence classes:

$$\{\{1\}, \{2\}, \{3\}\}$$

4 Make the Resulting Minimized DFA

We must now merge the indistinguishable states and determine the start state, transition function, and set of accepting states "in the standard way."

Identifying Indistinguishable States

After the algorithm terminates, all **empty cells** in the Indis Table (those with `True` values) represent pairs of states that cannot be distinguished by any input string. These states behave identically from the perspective of language acceptance.

Key insight: If $\text{Indis}(p, q) = \texttt{True}$ at the end, then for *every possible input string* w, either both p and q accept w or both reject w. Since they're functionally equivalent, we can merge them into a single state. **Final equivalence classes (states to be merged):**

$$\{\{1\}, \{2\}, \{3\}\}$$

Each equivalence class becomes a single state in the minimized DFA:

- States $\{1\}$ merge into state 1
- States $\{2\}$ merge into state 2
- States $\{3\}$ merge into state 3

Constructing the Minimized DFA

1. **States:** One state per equivalence class
2. **Start state:** The merged state containing the original start state
3. **Accepting states:** A merged state is accepting if any of its component states was accepting in the original DFA
4. **Transition function:** For each merged state and input symbol, pick any representative from the equivalence class, see where that state transitions, and the merged state transitions to whichever merged state contains that target

Result

The minimized DFA accepts exactly the same language as the original DFA but uses the minimum number of states possible. States that could not be distinguished by any input string have been merged while preserving the transition structure and accepting behavior.

Why This Works

- **Correctness:** Two states are merged only if no input string can distinguish them (empty cells in Indis Table)
- **Completeness:** The algorithm exhaustively checks all possible distinguishing strings through iterative refinement
- **Termination:** Guaranteed because cells can only change from `True` to `False` (never the reverse) and there are finitely many cells
- **Minimality:** The result has the fewest possible states while preserving the language — merging any states that are marked as distinguishable would change the language

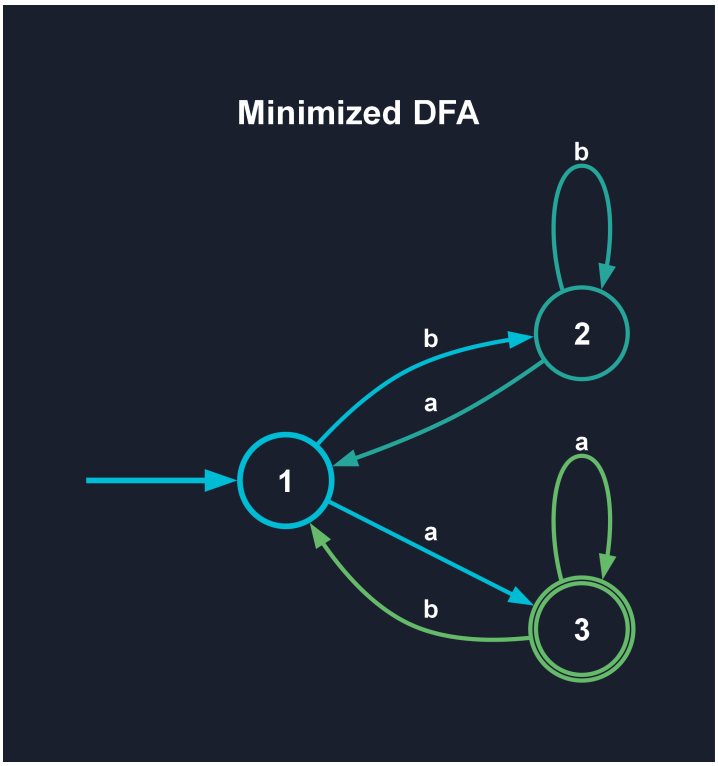

Figure 9: The minimized DFA with merged indistinguishable states